

SQL Mastery Course: Multi-CTE Queries, Window Functions & Optimization

Built for: Regulatory Data Associate → Risk Analyst / FRM track **Platform:** PostgreSQL
Pace: 2–3 hrs/week, self-paced (no fixed deadline) **Format:** 8 modules, each with concept notes, hands-on exercises, and a "prove it" checkpoint you must pass before moving on.

How this course works

- **No shortcuts to the checkpoint.** Each module ends with a checkpoint query you write from scratch, with no reference material open. If you can't do it cleanly, redo the exercises — don't peek at answers and call it learned.
 - **One dataset, used throughout.** You'll build a small insurance-flavored database once (Module 0) and reuse it for every module. This mirrors real regulatory/risk work: claims, policies, premiums, loss development.
 - **Insurance framing throughout.** Every exercise maps to something a Risk Analyst actually does: loss triangles, claim severity ranking, reserve trending, exposure aggregation. This isn't decoration — it's the vocabulary you'll need in interviews and on the job.
 - **Estimated time:** ~10–14 weeks at 2–3 hrs/week, but there's no clock. Move faster through what's easy, slower through window function frames (Module 5) — that's where most people plateau.
-

Module 0 — Environment & Practice Dataset (Week 1, ~2 hrs)

Goal: Get PostgreSQL running and load a realistic practice schema so every later module has real data to query.

Setup

1. Install PostgreSQL locally, **or** use a free hosted instance (Supabase, Neon, or ElephantSQL) if you don't want to manage a local install.
2. Install a GUI client: pgAdmin, DBeaver, or the free tier of TablePlus.

Build your practice database

Create these four tables — a simplified but realistic P&C insurance book:

```
sql
```

```

CREATE TABLE policies (
  policy_id      INT PRIMARY KEY,
  policyholder   TEXT,
  line_of_business TEXT,      -- e.g. 'Auto', 'Property', 'Liability'
  state         TEXT,
  effective_date DATE,
  expiration_date DATE,
  annual_premium NUMERIC(10,2)
);

CREATE TABLE claims (
  claim_id       INT PRIMARY KEY,
  policy_id      INT REFERENCES policies(policy_id),
  loss_date      DATE,
  report_date    DATE,
  claim_status   TEXT,        -- 'Open', 'Closed', 'Reopened'
  incurred_amount NUMERIC(10,2)
);

CREATE TABLE claim_payments (
  payment_id     INT PRIMARY KEY,
  claim_id       INT REFERENCES claims(claim_id),
  payment_date   DATE,
  payment_amount NUMERIC(10,2),
  payment_type   TEXT        -- 'Indemnity', 'Expense', 'Reserve Adjustment'
);

CREATE TABLE monthly_premiums (
  policy_id      INT REFERENCES policies(policy_id),
  month          DATE,
  earned_premium NUMERIC(10,2)
);

```

3. Populate with synthetic data. Ask me in a follow-up and I'll generate a realistic INSERT script (150-300 rows, believable date patterns, a few intentionally messy records) — that's a five-minute task once you're at this step.

Checkpoint

Run a basic query joining `policies` and `claims` to confirm data loaded correctly and relationships work. If this feels shaky, that's a signal to review basic JOINS before continuing — flag it and I'll fold in a Module 0.5.

Module 1 — Multi-CTE Fluency (Week 2-3, ~4-6 hrs)

You already know single CTEs. This module is about **chaining** them into a readable pipeline — the single biggest jump in query sophistication.

Concepts

- Sequential CTEs (each references the one before it)
- Why chained CTEs beat nested subqueries for readability and debugging
- Referencing an earlier CTE from a later one, multiple times
- Recursive CTEs (`WITH RECURSIVE`) — brief intro, mainly for hierarchy/date-spine problems

Exercises

1. Write a 3-CTE chain: (a) aggregate `claim_payments` by claim, (b) join to `claims` to get status/dates, (c) join to `policies` to add line of business — final SELECT reports total paid by line of business.
2. Rewrite a nested-subquery version of the same query, then compare readability. Note for yourself which one you'd want to debug at 5pm on a deadline.
3. Use `WITH RECURSIVE` to generate a calendar spine of months between two dates — useful for filling gaps in `monthly_premiums` (a real regulatory reporting problem: missing months must still show as zero, not be silently absent).

Checkpoint

From scratch: Write a 4-CTE query that produces, per line of business: total earned premium, total incurred losses, total paid losses, and a simple loss ratio (paid / earned). No looking at Exercise 1.

Module 2 — Window Function Fundamentals (Week 4, ~3 hrs)

Concepts

- The `OVER()` clause: what makes a window function different from a GROUP BY aggregate (rows aren't collapsed)
- `PARTITION BY` — resetting the calculation per group
- `ORDER BY` inside `OVER()` — how it changes what "the window" means
- Where window functions can appear in a query (SELECT list, not WHERE) and why

Exercises

1. For each claim, show its `incurred_amount` alongside the **average incurred amount for that policyholder's line of business** — without collapsing rows (this is the "aha" moment: same claim-level detail, extra context column).

2. For each policy, show total earned premium *and* that policy's share of its state's total earned premium.

Checkpoint

From scratch: For every claim, add a column showing how that claim's incurred amount compares to the average incurred amount for its `claim_status` category, in the same row-level detail.

Module 3 — Ranking Functions (Week 5, ~3 hrs)

Concepts

- `ROW_NUMBER()` vs `RANK()` vs `DENSE_RANK()` — the tie-handling differences (this trips almost everyone up once, so drill it deliberately)
- `NTILE(n)` for bucketing (e.g., splitting claims into quartiles by severity)
- Combining `PARTITION BY` with ranking for "top N per group" problems

Exercises

1. Rank claims by `incurred_amount` within each `line_of_business`, using all three ranking functions side by side on the same rows — deliberately pick a dataset with ties so you can see the difference concretely.
2. Classic "top N per group": find the 3 largest claims per state.
3. Use `NTILE(4)` to bucket policies into premium quartiles.

Checkpoint

From scratch: Identify the single largest open claim per line of business, using `ROW_NUMBER()` inside a CTE (this pattern — rank-then-filter-in-outer-query — is one you'll use constantly).

Module 4 — LAG / LEAD & Time-Series Comparisons (Week 6, ~3 hrs)

Concepts

- `LAG()` / `LEAD()` — pulling a prior/next row's value into the current row
- The `OFFSET` and `DEFAULT` arguments
- Why `ORDER BY` inside the window matters enormously here (get this wrong and results are silently nonsensical, not erroring)
- Month-over-month and period-over-period deltas — core regulatory reporting task

Exercises

1. For each policy's `monthly_premiums`, add a column showing the prior month's earned premium and the month-over-month change.

2. Flag any policy-month where earned premium dropped more than 20% from the prior month (a real "investigate this" trigger in regulatory data QA).
3. Use `LEAD()` to calculate days between a claim's `loss_date` and its next payment — a development-lag metric.

Checkpoint

From scratch: For each claim with multiple payments, show each payment alongside the **cumulative days since the loss date** and the **dollar change from the previous payment**. Two window functions, one query.

Module 5 — Aggregate Window Functions, Running Totals & Frames (Week 7-9, ~6-8 hrs)

This is the module where most people plateau. Budget extra time here.

Concepts

- `SUM()`, `AVG()`, `COUNT()`, `MIN()`, `MAX()` used as window functions
- Running totals: `SUM(x) OVER (ORDER BY ...)`
- **Frame clauses** — this is the part that's usually skipped and shouldn't be:
 - `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` (running total)
 - `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` (moving 3-period average)
 - `RANGE` vs `ROWS` — the difference matters when you have tied `ORDER BY` values, and insurance date data ties constantly (multiple claims same day)
- Moving averages — smoothing noisy monthly data, a genuinely common regulatory/actuarial task

Exercises

1. Running total of `payment_amount` per claim, ordered by `payment_date` — the classic "reserve draw-down" view.
2. 3-month moving average of `earned_premium` per policy, using an explicit `ROWS BETWEEN` frame.
3. Deliberately break a running total by forgetting `ORDER BY` inside the `OVER()` — observe the (wrong) result, then fix it. Doing this once on purpose saves hours of confusion later.
4. Build a simplified **loss development triangle**: cumulative incurred losses by policy, by month since loss date. This is a genuine actuarial artifact — expect it to take real effort.

Checkpoint

From scratch: Build a query showing, per line of business per month: total paid losses that month, and a running cumulative total of paid losses year-to-date. This combines GROUP

BY-style aggregation with a window function correctly — a common point of confusion (aggregate first, window second, in the right CTE).

Module 6 — Combining CTEs + Window Functions into Analytical Queries (Week 10-11, ~4-6 hrs)

Goal: Stop treating CTEs and window functions as separate topics. Real analytical queries chain both.

Exercises (each is a small "case")

1. **Loss ratio trend by line of business:** CTE to aggregate monthly premium and losses, then window function to show 3-month rolling loss ratio and its trend direction (`LAG` comparison).
2. **Claim severity outlier flagging:** CTE to compute average and standard deviation of incurred amounts by line of business, then flag claims more than 2 standard deviations above their group's mean.
3. **Reserve adequacy check:** for each claim, compare total paid-to-date against `incurred_amount` (the reserve estimate) and rank claims by how far they've overrun their reserve, within line of business.

Checkpoint

From scratch, no notes: Given the four tables, write one query that identifies the top 5 policies per state by loss ratio (paid losses / earned premium), where loss ratio is computed via CTE and ranking via window function. This mirrors a genuinely plausible take-home SQL test for a Risk Analyst role.

Module 7 — Query Optimization Basics (Week 12, ~3-4 hrs)

You don't need deep DBA-level tuning, but you should be able to read a query plan and reason about why a query is slow.

Concepts

- `EXPLAIN` and `EXPLAIN ANALYZE` — reading the output (Seq Scan vs Index Scan, cost estimates, actual vs estimated rows)
- When an index helps and when it doesn't — indexing columns used in `JOIN`, `WHERE`, `ORDER BY`
- Why `SELECT *` is a habit worth breaking, and why filtering early (in a CTE, before joining) beats filtering late
- Common anti-patterns: functions on indexed columns in `WHERE` (e.g. `WHERE DATE(loss_date) = ...` blocks index use), unnecessary `DISTINCT`, correlated subqueries where a window function would do

Exercises

1. Run `EXPLAIN ANALYZE` on your Module 6 checkpoint query. Identify the most expensive step.
2. Add an index on `claims.policy_id` and `claim_payments.claim_id`, re-run, compare plans.
3. Take a query using a correlated subquery and rewrite it with a window function — compare plans and (on larger synthetic data) execution time.

Checkpoint

From scratch: Given a slow query I'll hand you (deliberately containing 2-3 anti-patterns), explain what's wrong using `EXPLAIN ANALYZE` output and rewrite it.

Module 8 — Capstone (Week 13-14+, open-ended)

Build one end-to-end analytical report using everything above, unaided:

"Quarterly Book Review" — a single query (using CTEs and window functions throughout) that produces, per line of business:

- Written and earned premium, current quarter vs. prior quarter (LAG)
- Loss ratio and its rolling 3-month trend
- Top 3 largest open claims
- Any policy with a >25% premium drop quarter-over-quarter, flagged
- A running year-to-date cumulative paid-loss total

This is portfolio-worthy — write it up with a short explanation of your approach and keep it. It's a legitimate artifact to reference in an interview when asked "tell me about a SQL project you built."

Notes on pacing and getting unstuck

- If a checkpoint takes more than 2-3 attempts, that's data, not failure — it usually means the *previous* module's concept isn't fully solid yet. Better to drop back a module than push forward on a shaky foundation, especially before Module 5.
- Window function frames (Module 5) are where "beginner-intermediate" SQL usually gets stuck permanently, because most tutorials skip frame clauses entirely. Don't skip them here.
- When you're ready for Module 0's data, or want me to generate the synthetic dataset, or want feedback on a checkpoint query — just bring it to me directly; I can review your SQL and point out exactly what to fix rather than just giving you the answer.